

Database Schema Reference

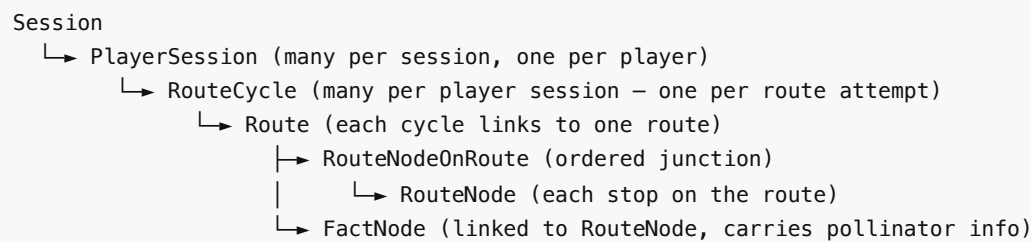
The backend uses **Prisma ORM** with two schema variants:

- [backend/prisma/schema.prisma](#) — MySQL/MariaDB (development default)
- [backend/prisma/postgresql/schema.prisma](#) — PostgreSQL (production recommended)

The schemas are functionally identical. The only differences are the `provider` field and `@db.LongText` vs `@db.Text` for large text fields.

See [MYSQL LICENSE ISSUE.md](#) for why PostgreSQL is recommended for production.

Entity Relationship Overview



Tables

Session

Represents one event day at the park. Created by IT staff before each event, or automatically via `/api/create-session`.

Column	Type	Constraints	Description
id	Int	PK, auto-increment	Internal ID (not the session code)
sessionId	Int	Unique	The 9-digit public session code (e.g. 202604150)
sessionDate	DateTime	Indexed	The calendar date the session is for
Notes:			

- `sessionId` is the value players scan via QR code.
- `sessionDate` is used by the JWT validation (`hasession.service.ts`) to verify the session is valid for today (Eastern timezone).
- Sessions are pre-seeded for Jan–Dec 2024 with IDs in `YYYYMMDD0` format.

PlayerSession

Represents one player's participation in one session. Created the first time a player calls `/api/start-game`.

Column	Type	Constraints	Description
--------	------	-------------	-------------

id	Int	PK, auto-increment	Internal player-session ID
sessionId	Int	FK → Session.sessionId	The session this player is in
playerId	Int		The player's device ID (generated client-side, stored in JWT)
startTime	DateTime		When the player first joined (set on creation)
numChildren	Int?	Nullable	Children count from party size survey
numAdults	Int?	Nullable	Adults count from party size survey

Unique constraint: (sessionId, playerId) — one record per player per session.

Notes:

- numChildren and numAdults are null until the player submits the survey.
- The unique constraint means createMany({ skipDuplicates: true }) is used for idempotent creation.
- playerId range: 1 to 2,000,000,000 (generated by generateUniquePlayerId() in api.utilities.ts).

RouteCycle

Records a single route assignment for a player. A new row is created each time a player is assigned a route (whether new or replay).

Column	Type	Constraints	Description
id	Int	PK, auto-increment	Internal ID
sessionId	Int	FK → Session.sessionId	Denormalized for query performance
playerSessionId	Int	FK → PlayerSession.id	The player-session this belongs to
routeId	Int	FK → Route.id	Which route was assigned
cycleNumber	Int	Default 1	1 = first pass through all routes, 2 = first replay, etc.
startedAt	DateTime		When the route was assigned
completedAt	DateTime?	Nullable	When the route was completed
completionMs	Int?	Nullable	Duration of completion in milliseconds
wasCompleted	Boolean	Default false	Whether the player finished the route
meta	Json?	Nullable	Reserved field — never populated

Unique constraint: (playerSessionId, routeId, cycleNumber) — prevents duplicate assignments.

Notes:

- A player who receives all 11 routes gets `cycleNumber = 2` on their next assignment (replay).
- `wasCompleted = false` means the player is still on the route OR abandoned it.
- `completedAt` and `completionMs` are set when `POST /api/complete-route` is called.
- `meta` is a dead-code field — never read or written.

Route

Master data record for each pollinator route. Seeded at startup; not modified at runtime.

Column	Type	Constraints	Description
<code>id</code>	Int	PK, auto-increment	Internal route ID
<code>name</code>	String	Unique	Pollinator name (e.g. "Bee", "Hummingbird")
<code>sourcePath</code>	Json?	Nullable	Legacy field from old route format — can be ignored

Notes:

- 11 routes are seeded: Human, Bat, Hummingbird, Sunbird, Moth, Butterfly, Fly, Bee, Wasp, Beetle, Ant.
- `name` maps to the pollinator displayed at the end of a route.
- Route content (stops and facts) is stored in `RouteNode` , `RouteNodeOnRoute` , and `FactNode` .

RouteNode

Represents a single stop card on a route — the text shown at a specific location.

Column	Type	Constraints	Description
<code>id</code>	Int	PK, auto-increment	Internal ID
<code>key</code>	String	Unique	Spritesheet coordinate key (e.g. "1.0", "3.2")
<code>text</code>	String (LongText/Text)		The interpretive content shown at this stop

Notes:

- `key` encodes both the row and column in the spritesheet (format: `"row.column"`).
- A single `RouteNode` can appear on multiple routes (shared stops).
- See [ADDING ROUTES.md](#) for the coordinate system.

RouteNodeOnRoute

Junction table connecting routes to their ordered stops.

Column	Type	Constraints	Description
<code>routeId</code>	Int	FK → Route.id, part of composite PK	

routeNodeId	Int	FK → RouteNode.id, part of composite PK	
position	Int		Order of this stop within the route (0-indexed)

Composite PK: (routeId, routeNodeId)

Notes:

- Querying `RouteNodeOnRoute WHERE routeId = X ORDER BY position` gives the ordered stops for route X.
- Used by `LoadRouteDT0.service.ts` to assemble the full route.

FactNode

A fact card linked to a specific `RouteNode`, identifying which pollinator it's about.

Column	Type	Constraints	Description
id	Int	PK, auto-increment	Internal ID
nodeId	String	Unique, FK → RouteNode.key	The route node this fact belongs to
text	String (LongText/Text)		The fact text shown after the stop card
pollinator	String		The pollinator this fact is about

Notes:

- `nodeId` references `RouteNode.key` (not `RouteNode.id`) — a string key like `"4.2"`.
- Every fact node's `pollinator` field determines what's displayed in the collection screen.
- A route's "reveal" pollinator comes from the last fact node's `pollinator` field.

Migrations

Three migrations exist in `backend/prisma/migrations/`:

20260224061121_init

Initial schema — creates all 7 tables with relationships, indexes, and cascades.

Key relationships established:

- `PlayerSession.sessionId` → `Session.sessionId` (restrict delete)
- `RouteCycle.playerSessionId` → `PlayerSession.id` (cascade delete)
- `RouteCycle.routeId` → `Route.id` (restrict delete)
- `RouteNodeOnRoute.routeId` → `Route.id` (cascade delete)
- `RouteNodeOnRoute.routeNodeId` → `RouteNode.id` (restrict delete)
- `FactNode.nodeId` → `RouteNode.key` (set null on delete)

20260228120000_add_player_session_num_children_adults

Adds `numChildren Int?` and `numAdults Int?` to `PlayerSession`. These support the party size survey feature.

20260316200703_add_survey_fields

No-op migration. This was a duplicate of the previous migration (the survey fields were already added). The file exists for migration history integrity but contains no schema changes.

Prisma Client

The backend uses a **runtime adapter selection** pattern in `backend/src/services/db/prisma.ts`:

```
DATABASE_URL starts with "postgresql://" → use @prisma/adapter-pg
DATABASE_URL starts with "mysql://"      → use @prisma/adapter-mariadb
```

Both adapters are installed and the Prisma client is generated at build time. The selection happens at runtime based on the `DATABASE_URL` environment variable.

Prisma client generation:

- MySQL variant: `npx prisma generate (uses prisma/schema.prisma)`
- PostgreSQL variant: `npx prisma generate --config prisma.config.pg.ts (uses prisma/postgresql/schema.prisma)`

Both generate to `backend/generated/prisma/` — do not commit this folder.

Seed Data

The database is seeded on first startup by `prisma/seed.js` (MySQL) or `prisma/seed.pg.js` (PostgreSQL). The seed creates:

- **Sessions:** One session per day from January 1, 2024 to December 31, 2024 (ID format: `YYYYMMDD0`)
- **Routes:** 11 route master records
- **RouteNodes:** All unique stop nodes (shared across routes)
- **RouteNodeOnRoute:** Ordered junction records for all 11 routes
- **FactNodes:** One fact per terminal node, each with a `pollinator` field

For the coordinate system and how to add routes, see [ADDING ROUTES.md](#).